Render Clone + CI/CD - Project Brief

Team 3: Aditya & Sneha

🎯 The Problem

Students build apps but struggle to deploy them. Cloud platforms cost money. Their own hardware (laptop, Raspberry Pi) sits idle.

Your Mission: Build a platform where users can deploy their apps on their own hardware with automatic CI/CD, like Render but using personal devices.

✅ Must Have (Core Features)

1. User Dashboard

- Register and login
- See all deployed apps
- View deployment status and logs

2. Connect Your Hardware

- Users run an "agent" on their laptop/Raspberry Pi/server
- Agent connects to your platform
- Shows hardware status (online/offline, CPU, memory)

3. Deploy from GitHub

- User provides GitHub repo URL
- System pulls code automatically
- Detects what type of app (Node.js, Python, static site, etc.)
- Runs it on user's connected hardware

4. Basic CI/CD

- When user pushes to GitHub, auto-redeploy
- Show build logs in real-time
- Success/failure notifications

5. App Management

- Start/stop/restart apps
- View logs
- Set environment variables
- Delete deployments

🎨 Your Creative Freedom

You Decide:

- How the agent communicates with your platform
- UI/UX design
- What programming languages/frameworks to support
- How to handle multiple hardware devices
- Deployment workflow
- Log viewing interface

Go Beyond (Optional Ideas):

- Support Docker containers
- Custom domains
- SSL certificates
- Database deployment
- Load balancing across multiple devices
- Resource usage graphs
- Rollback to previous versions
- Scheduled deployments
- Webhook integration

- Slack/Discord notifications
- Multiple environments (dev, staging, prod)

🤖 AI Integration (Required)

Minimum: AI helps with deployment configuration

Ideas to Explore:

- Detect project type and suggest build commands
- Analyze errors and suggest fixes
- Optimize deployment settings
- Predict resource requirements
- Generate Dockerfile automatically
- Troubleshoot failed deployments
- Suggest performance improvements

Example AI Usage:

```
User uploads Node.js repo
AI detects: "This is an Express.js app"
AI suggests:
  Build: npm install
  Start: npm start
  Port: 3000
  Environment needs: Node.js 18+
```

AI Tools: OpenAI, Gemini, Claude

🔧 Technical Concepts

Agent Architecture

The agent runs on user's hardware and:

- Registers with your central server
- Reports hardware status

- Receives deployment commands
- Executes builds
- Streams logs back to server

Communication

- WebSockets for real-time updates
- REST API for commands
- GitHub webhooks for CI/CD

Simple Flow

```
1. User adds GitHub repo URL in dashboard
2. Platform sends deploy command to agent
3. Agent clones repo on local hardware
4. Agent runs build commands
5. Agent starts the app
6. Platform shows app as "Running"
```

📦 What to Submit

1. Working Platform

- Deployed web dashboard
- Downloadable agent (executable or script)
- At least one successful deployment demo

2. Documentation

- **README.md:** How everything works
- **Agent Setup Guide:** How to install and run agent
- **Deployment Guide:** How to deploy an app
- **Postman Collection:** API endpoints
- Architecture diagram (simple drawing is fine)

3. Marketing Stuff

- Product Hunt launch plan
- Screenshots
- Demo video showing deployment process

💡 Quick Tips

Start Simple:

1. Build basic dashboard with auth
2. Create simple agent that reports status
3. Add manual deploy (user clicks button)
4. Then add GitHub integration
5. Finally add auto-redeploy on push

Agent Can Be:

- Python script
- Node.js app
- Go binary
- Whatever you're comfortable with

Test With:

- Deploy a simple Express.js "Hello World"
- Deploy a static HTML site
- Then try more complex apps

Tech Suggestions:

- Dashboard: React/Vue + Node.js backend
- Agent: Python, Node.js, or Go
- Communication: WebSockets (Socket.io)
- Database: PostgreSQL or MongoDB
- Queue: Redis (for job management)

Focus On:

- Getting one deployment working perfectly

- Real-time log streaming
- Agent stability
- Good error messages

⏰ Timeline: 7 Days

Suggested Breakdown:

- Days 1-2: Dashboard + agent skeleton + connection
- Days 3-4: GitHub integration + basic deploy
- Day 5: CI/CD + auto-redeploy + AI integration
- Day 6: Deploy + documentation
- Day 7: Marketing + demo video

⚠️ Important Notes

Security:

- Agent should authenticate with platform
- Don't expose user's hardware to internet directly
- Validate GitHub webhooks
- Secure environment variables

Hardware Support:

- Start with Linux/Mac (easier)
- Windows optional
- Raspberry Pi would be cool to demo!

Scope:

- Don't try to compete with Render fully
- Focus on the core idea working well
- One supported runtime (Node.js) is fine
- Can expand later

✨ What Makes This Great

Your project stands out if:

- Agent installation is simple
- Deployment actually works reliably
- Logs are clear and helpful
- UI is clean and informative
- You can demo it live on your own hardware
- AI genuinely helps with setup/troubleshooting

Remember: The coolest part is using your own hardware! Make sure that works smoothly. Better to support Node.js apps really well than 10 languages poorly.

Good luck! 🚀